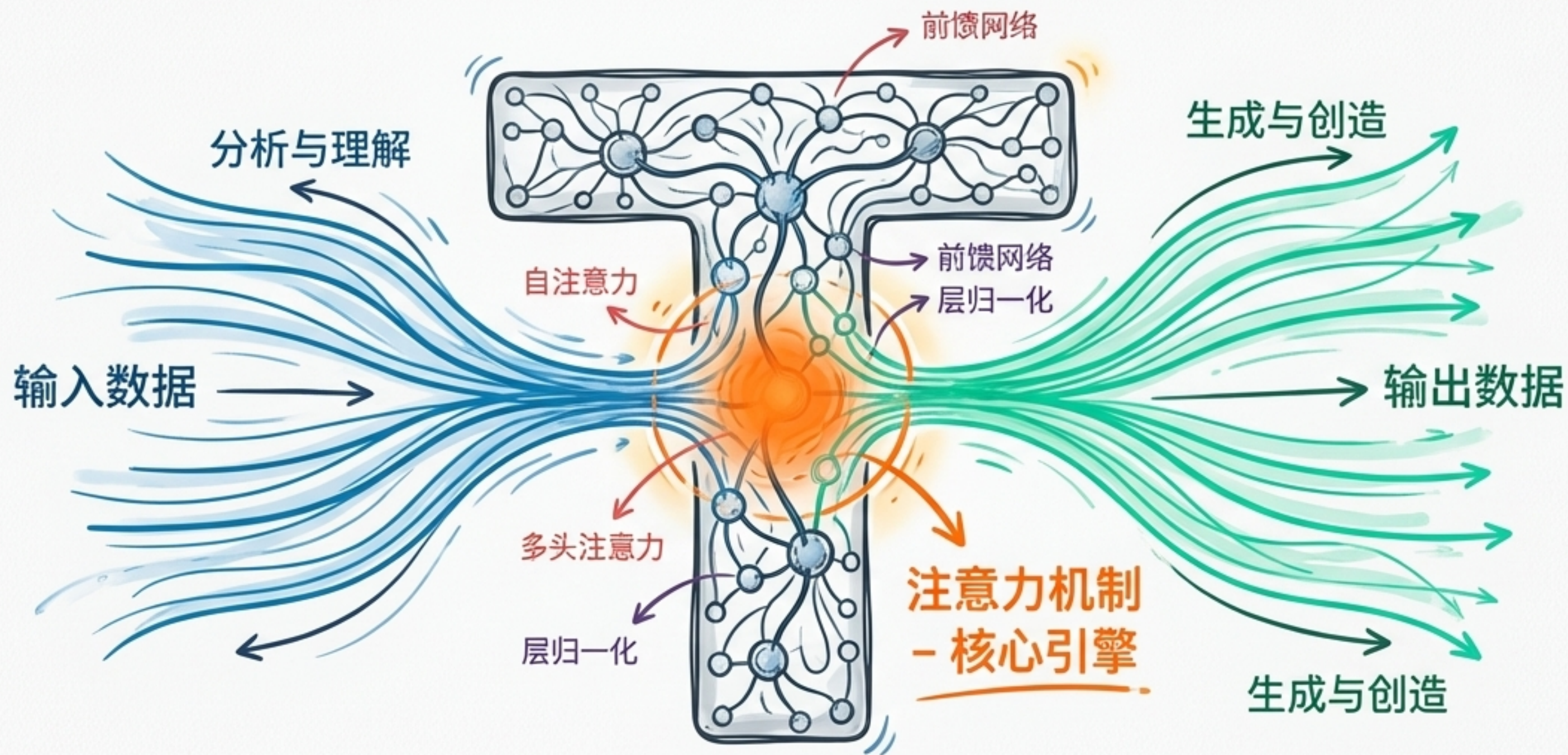


Transformer核心原理与训练优化

一场颠覆NLP（甚至整个AI领域）的架构革命



我们的探索之旅



传统序列模型的“三座大山”

传统RNN的瓶颈



梯度消失/爆炸

随序列长度增加，信号在反向传播中指数级衰减或增长，难以训练。



难以并行

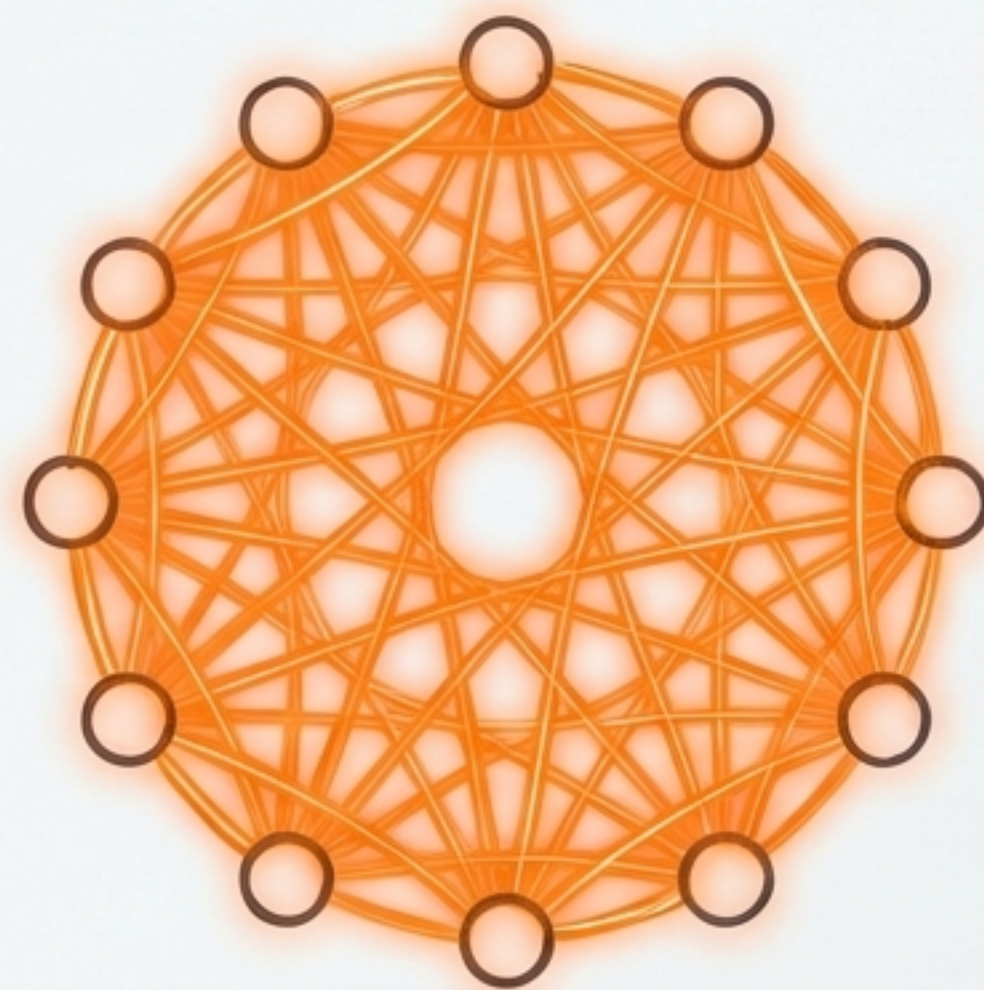
时间步计算存在依赖，必须串行处理，无法充分利用现代GPU的并行计算能力。



长距离记忆衰减

关键信息在长序列传递中容易丢失，难以捕捉长期依赖。

Transformer的解决方案

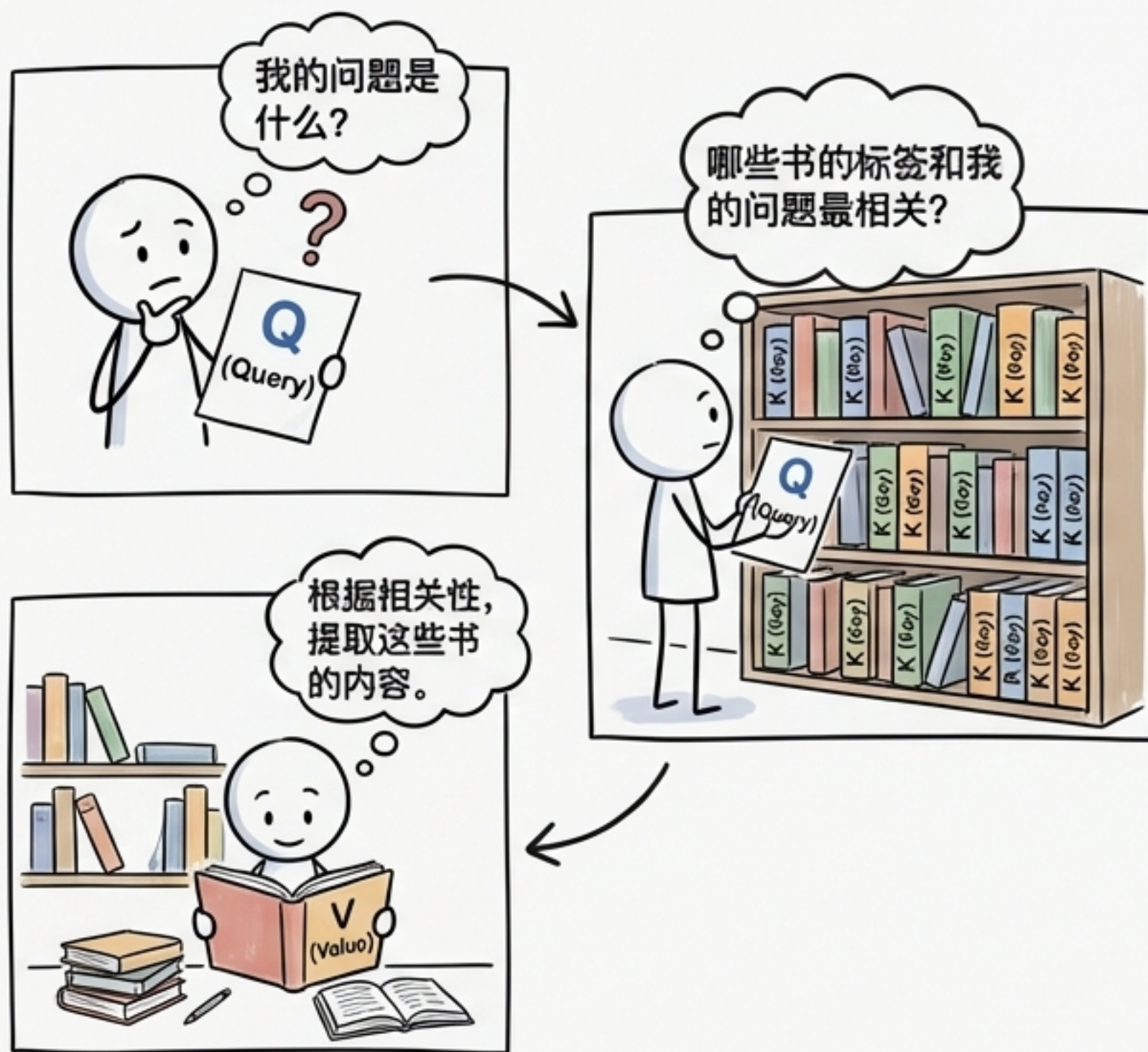


通过自注意力机制，一次性扫描全局，摆脱递归。

实现并行化与长程依赖无损传递，奠定现代大模型基础。

自注意力机制的核心：QKV模型与计算

将全局上下文融入每个位置的表示



三步计算流程

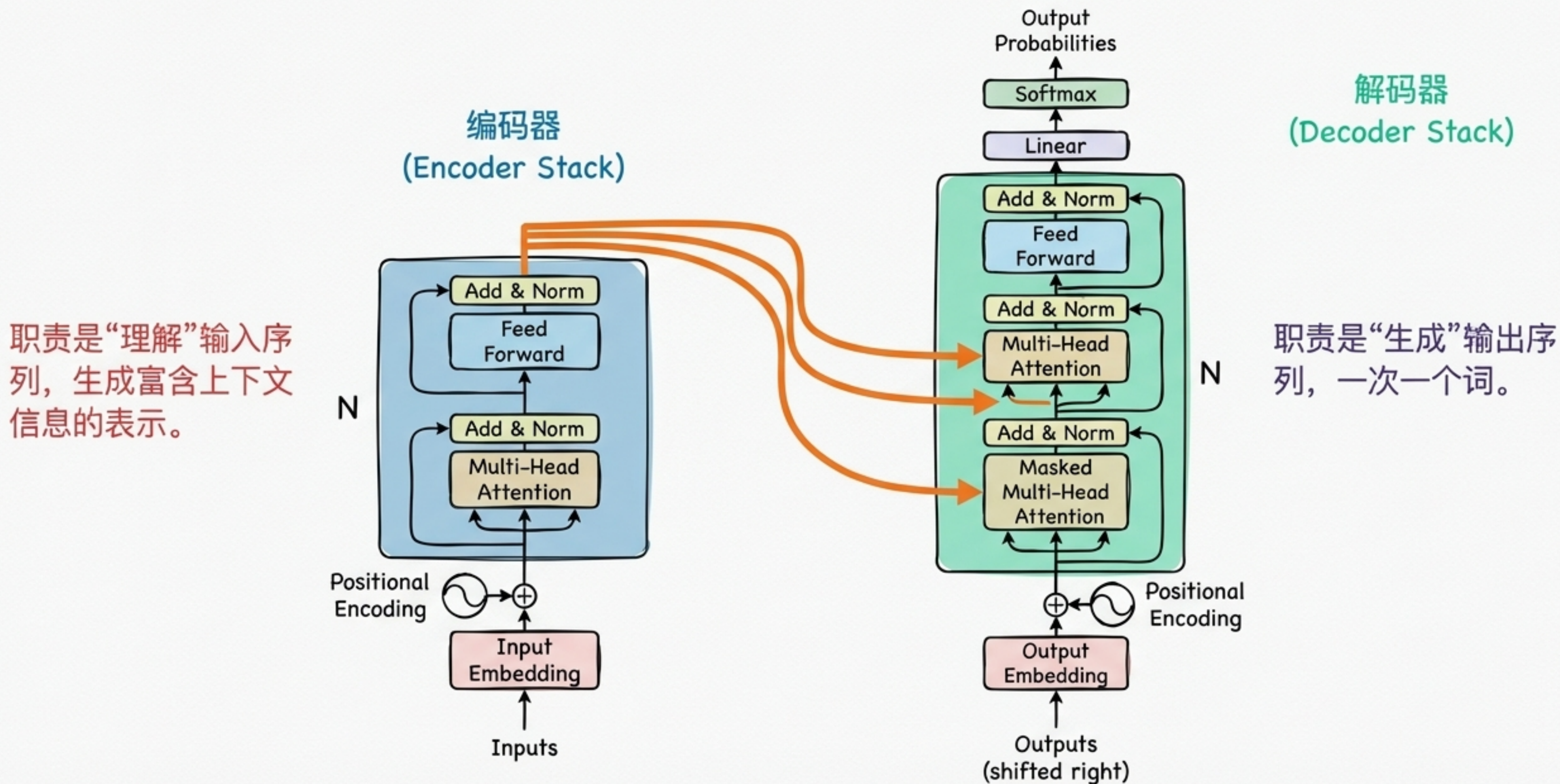
- 线性映射
对输入 X 分别乘以权重矩阵 W_q , W_k , W_v , 得到查询 (Q)、键 (K)、值 (V) 矩阵。
- 计算注意力权重
通过 Q 与 K 的点积计算相关性分数, 并进行归一化。
- 加权求和
使用注意力权重对 V 进行加权求和, 得到包含全局信息的输出表示。

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{Q * K^T}{\sqrt{d_k}} \right) * V$$

Diagram illustrating the Attention formula with annotations:

- 计算相似度 (Calculate Similarity) points to $Q * K^T$.
- 归一化权重 (Normalize Weights) points to the softmax function.
- 防止梯度过小 (Prevent gradient from being too small) points to $\sqrt{d_k}$.
- 加权求和 (Weighted Sum) points to the final multiplication $* V$.

Transformer 整体架构：编码器-解码器



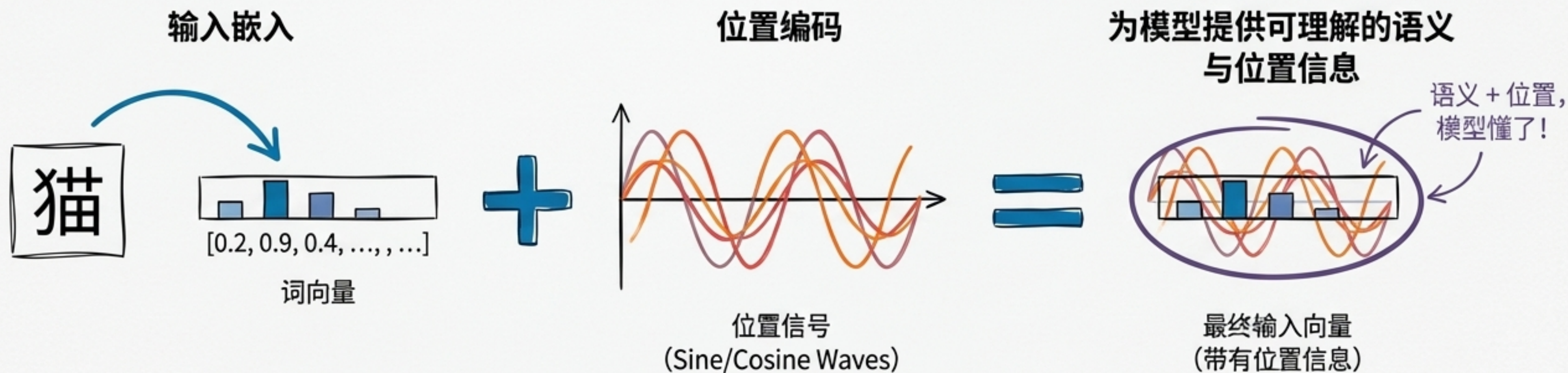
编码器第一步：如何让模型理解“词”与“位置”？

输入嵌入 (Input Embedding)

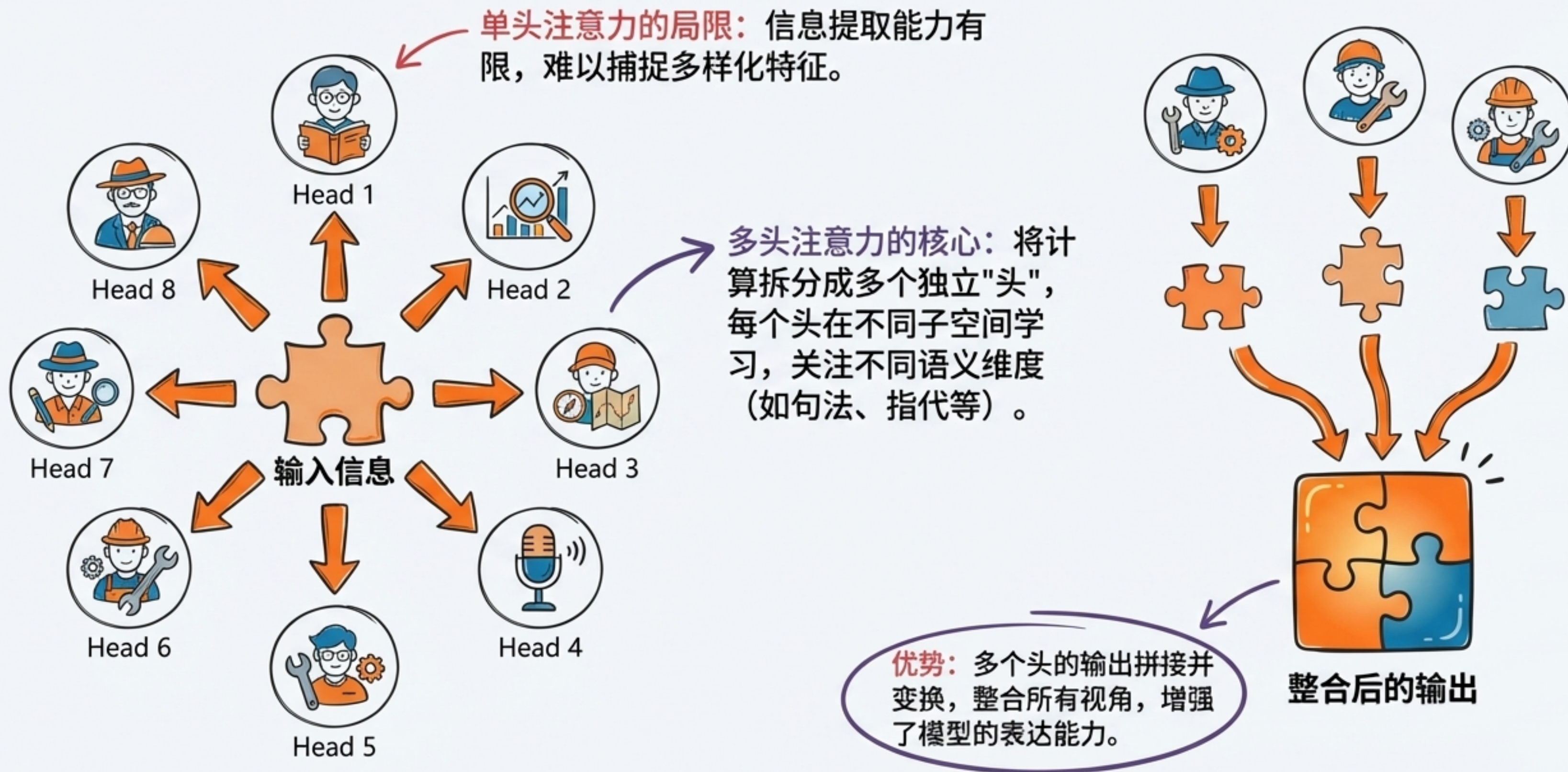
- • **作用：**将离散的词元转换为连续的稠密向量，捕捉语义信息。
- **类比：**为每个汉字/单词生成一个独特的“身份证号”（向量）。

位置编码 (Positional Encoding)

- ② **为什么需要？** Transformer并行处理所有词，不像RNN那样天然具有顺序信息。没有位置信息，“猫抓老鼠”和“老鼠抓猫”会被视为相同意思。 重要！
- 🔊 **如何实现？** 使用正弦/余弦函数为每个位置注入一个独特的、固定的位置信号，叠加到词嵌入上。

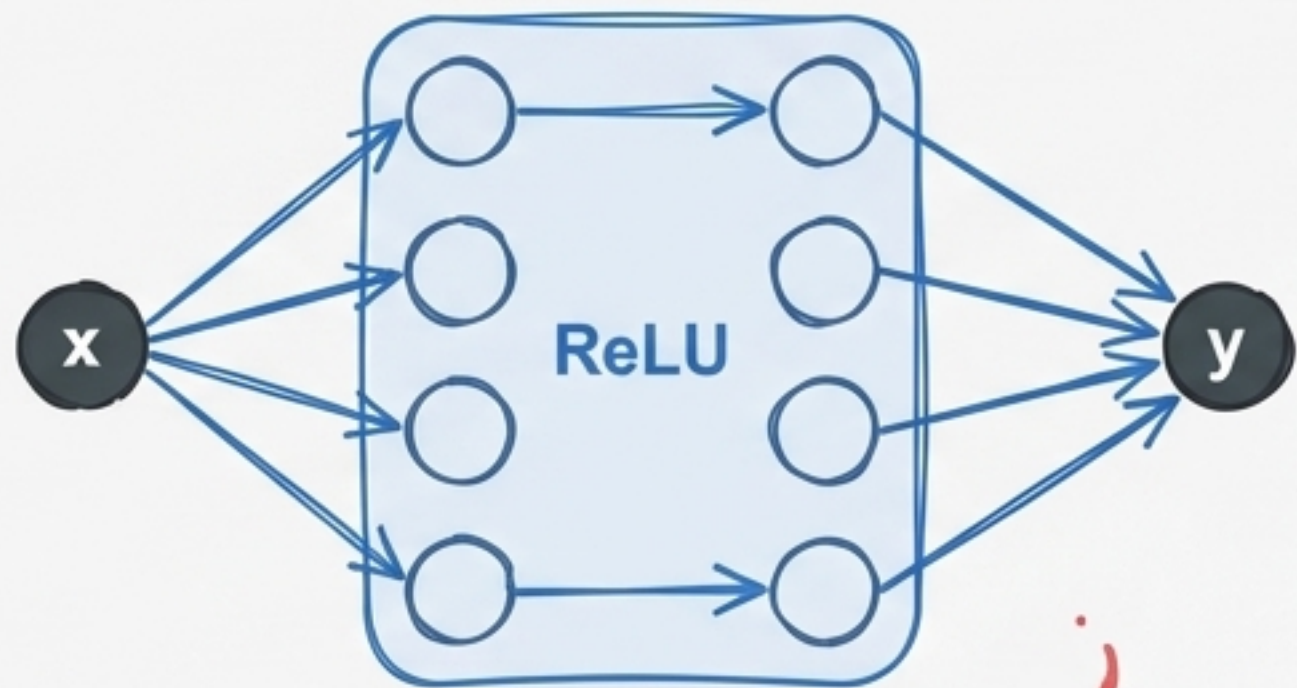


多头自注意力：从“单一视角”到“专家会诊”



FFN与残差连接：深度加工与信息高速路

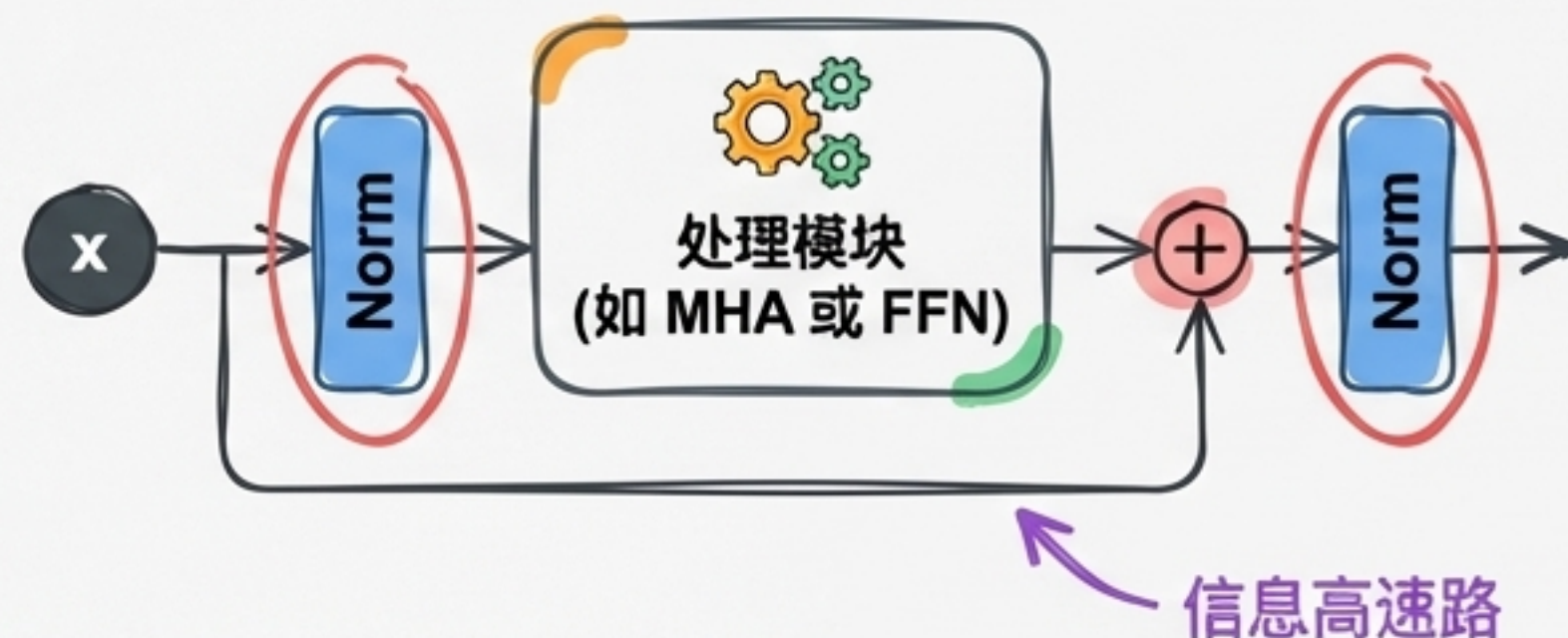
位置前馈网络 (FFN)



作用：对注意力输出进行非线性变换，提取更高阶的特征。自注意力负责“联系”所有相关词，FFN则对联系好的信息进行“深度加工”。

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

残差连接 (Residual Connection) & 层归一化 (Layer Norm)



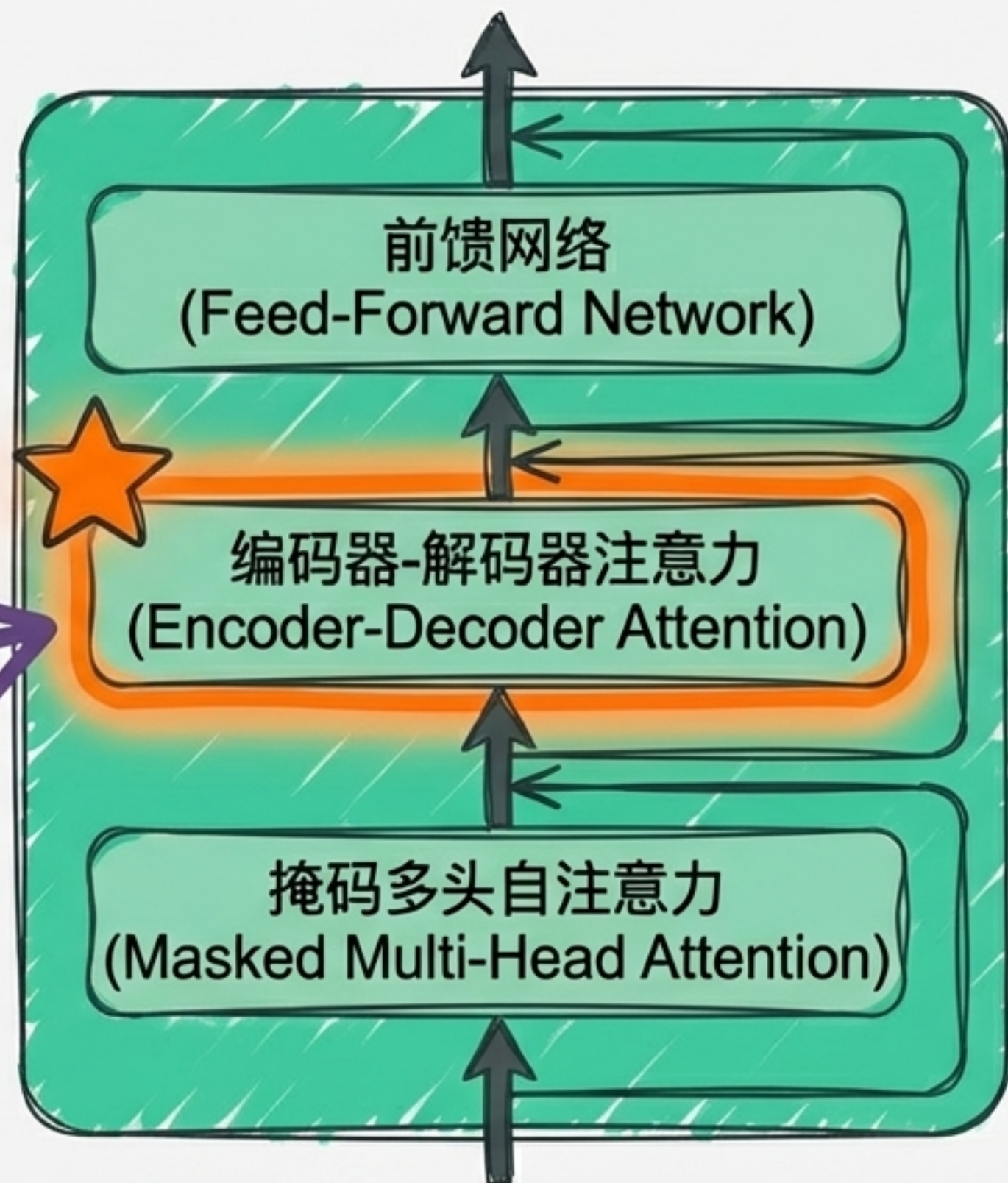
残差连接：解决深层网络中的信息稀释问题。把‘原始输入’和‘加工结果’加在一起，确保原始信息不丢失。就像开辟了一条‘信息高速路’。

层归一化：稳定训练过程。对每一层的输入进行标准化处理，缓解梯度消失，加速模型收敛。

解码器架构：比编码器多出的“秘密武器”

- 解码器核心单元的三个子层：
- 1. 掩码多头自注意力 (Masked Multi-Head Attention)
- 2. 编码器-解码器注意力 (Encoder-Decoder Attention)
——这是解码器独有的“秘密武器”
- 3. 前馈网络 (Feed-Forward Network)

关键区别：连接
理解与生成的桥梁



解码器的魔法：掩码与交叉注意力

掩码自注意力（Masked Self-Attention）

- 目的：确保生成顺序性，防止“偷看未来”。
- 机制：通过一个上三角矩阵的掩码，确保在预测第 i 个词时，只能关注到 $i-1$ 及之前的所有位置。

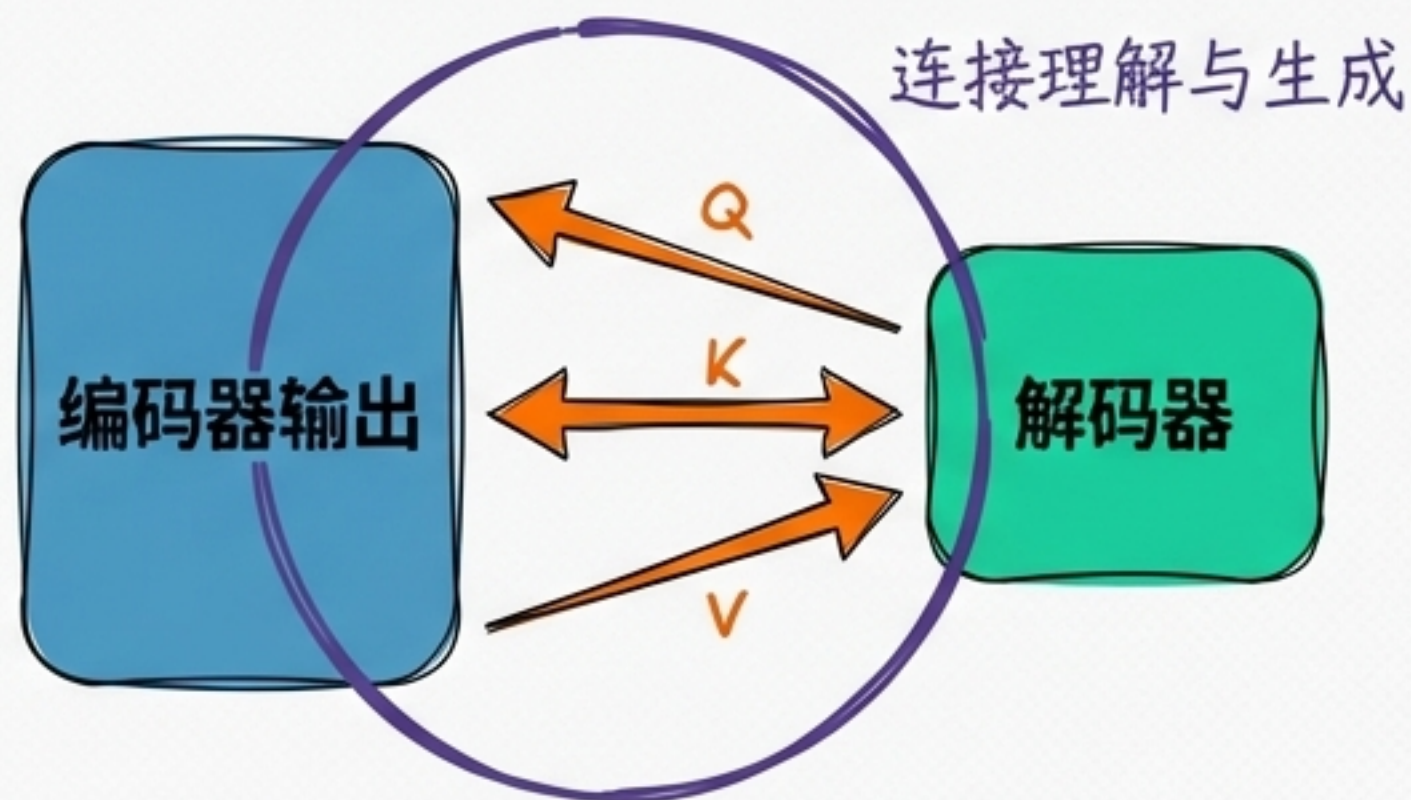
Attention Scores

		×	×	×	×
2	4	×	×	×	×
	4	1	×	×	×
3		2	3	×	×
3	5		1	2	

不能偷看未来

交叉注意力（Cross-Attention）

- 目的：实现源序列与目标序列的信息交互。
- 机制：Query (Q) 来自解码器自身，而 Key (K) 和 Value (V) 来自编码器的最终输出。这相当于解码器在生成每个词时，都在向编码器“提问”：“源序列的哪部分信息对我现在最重要？”



最后一步：线性层与Softmax - 选出最可能的词

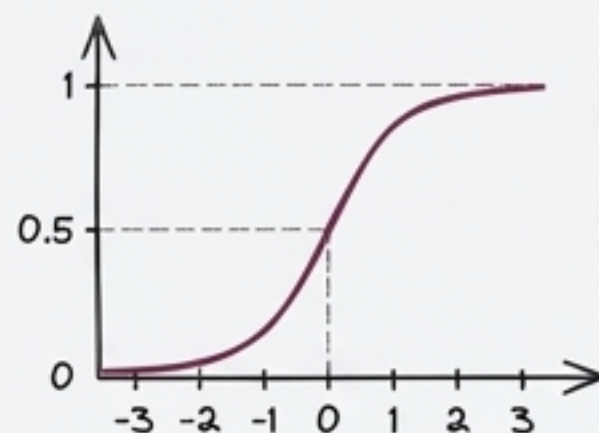
线性投影 (Linear Projection)



解码器输出向量

将解码器输出的隐藏状态向量，投影到整个词汇表大小的维度上，得到一个logits向量。每个分量代表对应词汇的分数。

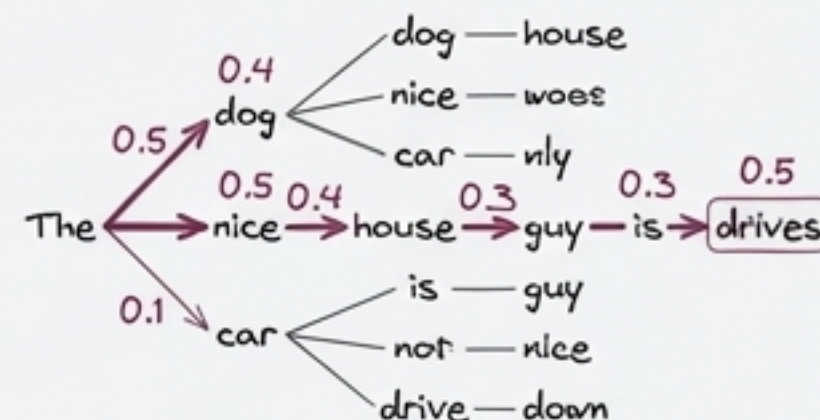
Softmax



Logits -> 概率分布

将logits向量转换为一个概率分布，所有分量和为1，每个分量代表对应词汇是下一个词的概率。

采样/搜索 (Sampling/Search)



选出最终词汇

根据概率分布选择最终的输出词。常用策略如Beam Search，用于生成更连贯的序列。

优化目标：交叉熵损失与标签平滑

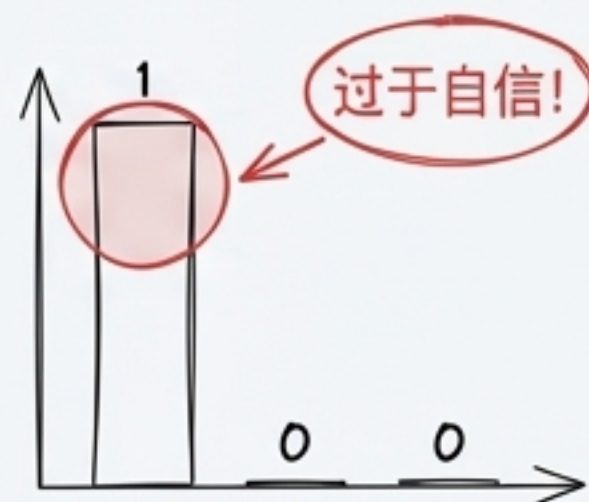
交叉熵损失 (Cross-Entropy Loss)



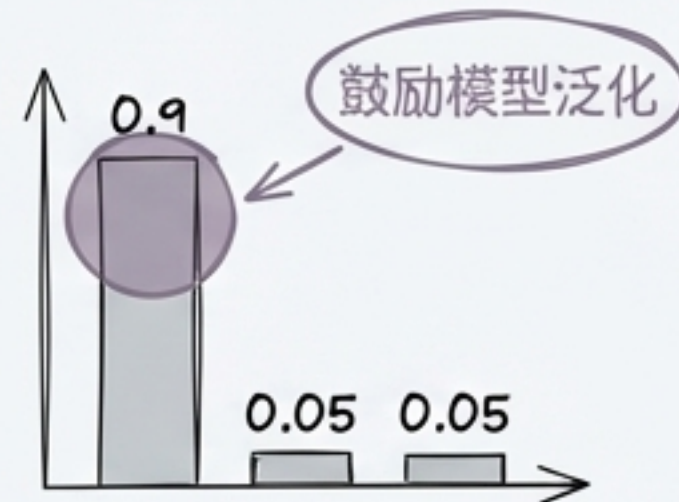
作用：衡量模型预测的概率分布与真实的“one-hot”标签分布之间的差异。是训练的直接优化目标。

标签平滑 (Label Smoothing)

Before: One-Hot 标签



After: 平滑后标签



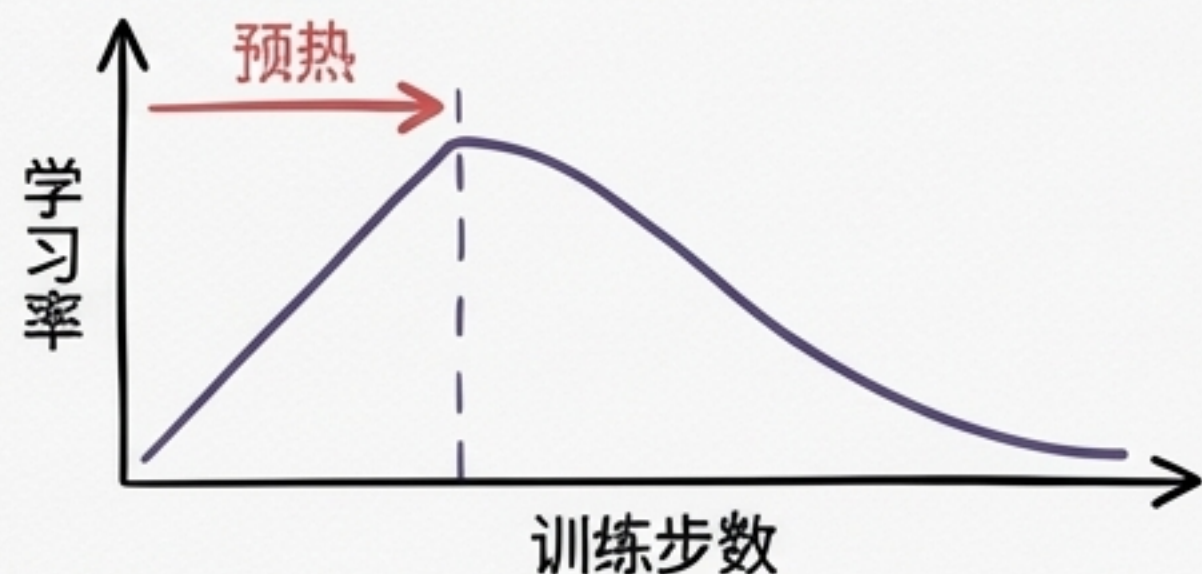
问题：使用绝对的one-hot标签会让模型变得“过于自信”，可能导致过拟合。

解决方案：将正确的标签概率从1略微降低（如0.9），并将其余的概率均匀分配给其他词。

效果：起到正则化作用，提升模型的泛化能力。

高效训练：学习率调度与工程优化

学习率调度 (Learning Rate Scheduling)



重要性：对于Transformer，固定的学习率很难使其稳定收敛。

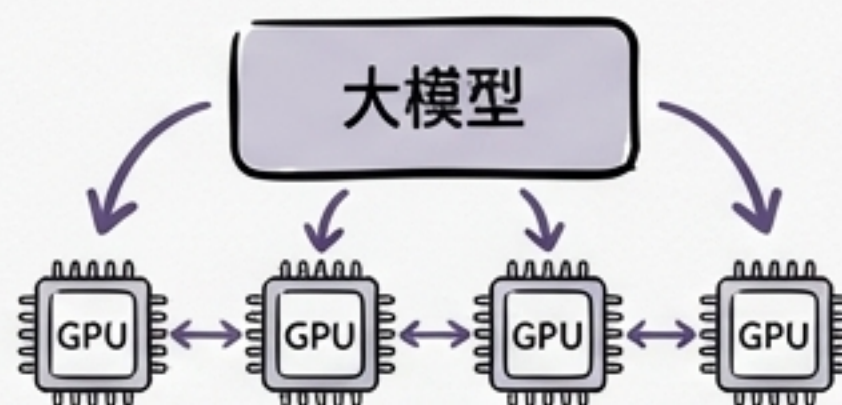
重要性：对于Transformer，固定的学习率很难使其稳定收敛。

常用策略：采用带有“预热”的策略，学习率先从小到大线性增长，达到峰值后再逐渐衰减，有助于在训练初期稳定模型，后期精细调优。

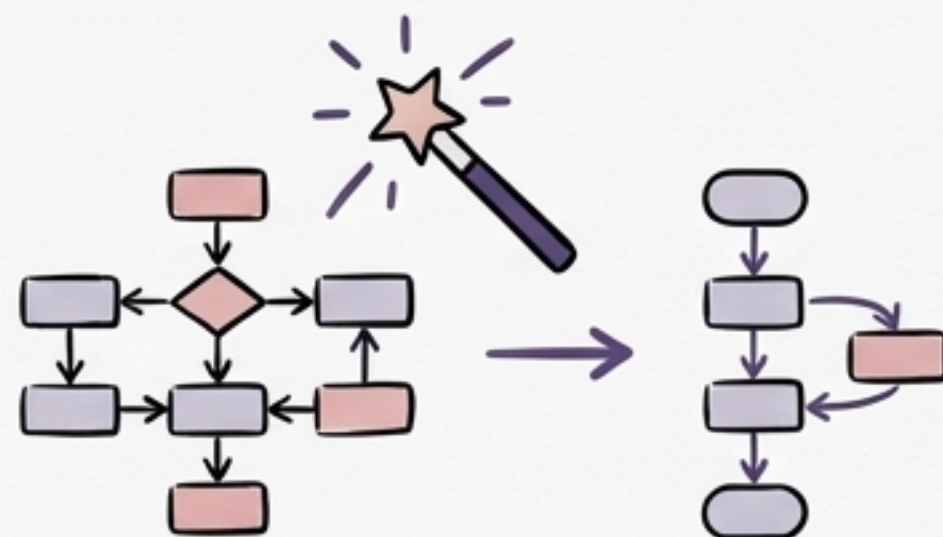
训练加速与硬件适配



混合精度训练 (Mixed-Precision)：使用半精度浮点数 (FP16) 进行计算，减少内存占用和计算时间。



并行计算：利用张量/流水线并行等技术，将大模型和数据分布到多个GPU上训练。



推理优化：通过KV缓存、算子融合等工程手段，降低推理延迟，提升吞吐量。

Transformer的演进与核心贡献

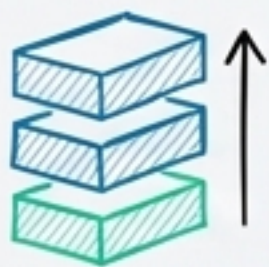
核心贡献回顾



- **并行化是基石**：自注意力机制摆脱了RNN的序列依赖。

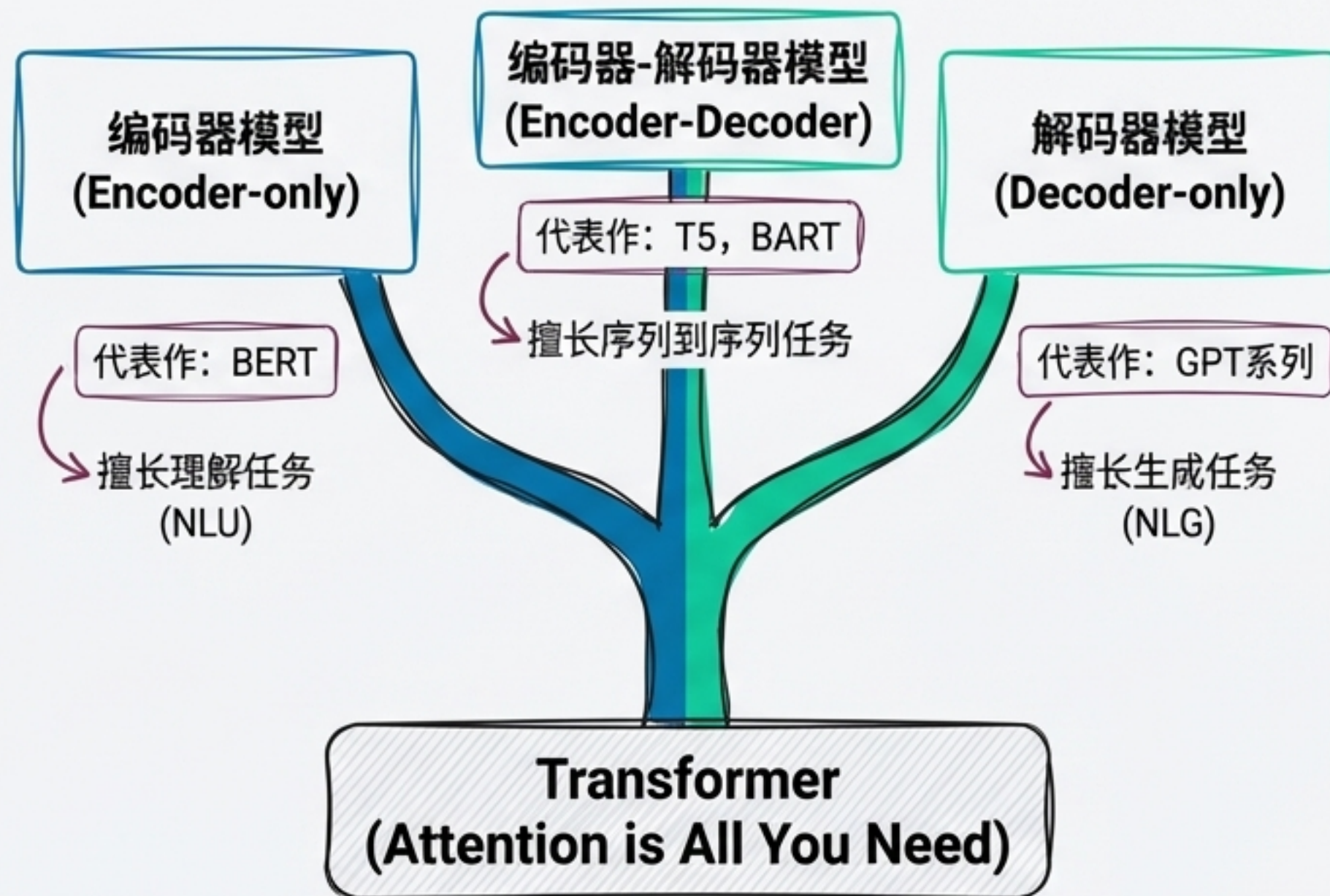


- **全局信息交互**：完美捕获长距离依赖。



- **堆叠构建深度**：通过堆叠Encoder/Decoder层构建强大的表征能力。

Transformer“家族”



未来方向：下一代架构的探索

- **效率提升**：探索稀疏/线性化注意力 (Sparse/Linear Attention) 等更高效的计算模式，以解决平方复杂度问题。
- **更长上下文处理**：研究记忆机制 (Memory Mechanisms) 等方法，让模型能处理书籍、代码库等超长序列。
- **迈向通用智能**：将Transformer架构扩展到多模态领域，统一处理文本、图像、语音等多种信息。

